



深圳大学  
Shenzhen University

# 互联网编程

## 第4章 Internet地址

深圳大学计算机与软件学院

# 主要内容

- **Internet地址概念**
- **DNS及与IP地址关系**
- **InetAddress类及应用编程**
- **InetAddress对象与程序安全性问题**
- **DNS缓存应用编程**
- **InetAddress的主机名和域名获取方法及应用编程**
- **测试IP地址性质编程**
- **NetworkInterface类及编程应用**
- **InetAddress的垃圾邮件检测编程应用（SpamCheck）**
- **InetAddress的离线处理日志文件编程应用**
- **编程应用思考与练习**



## Internet索引（IP地址）

- 所有连入Internet的终端设备(包括计算机、PDA、打印机以及其他的电子设备)都有一个唯一的索引，这个索引被称为**IP地址**。
- 在Internet上的IP地址大多由**4个字节**组成，这种IP地址叫做**IPv4**。
- 除了这种由四个字节组成的IP，在Internet上还存在一种IP，这种IP由**16个字节**组成，叫做**IPv6**。
- IPv4和IPv6后面的数字是Internet协议(Internet Protocol, IP)的版本号。



## IPv4与IPv6混合网络

- 在IPv4和IPv6混合的网络中，IPv6地址的后四个字节可以被写成IPv4的地址格式。
- 如FEDC:BA98:7654:3210:FEDC:BA98:7654:3210可以写成FEDC:BA98:7654:3210:FEDC:BA98:118.84.50.16。
- 当访问网络资源的计算机使用的是IPv4的地址时，系统会自动使用IPv6的后四个字节作为IPv4的地址。





# DNS

- 无论是IPv4地址，还是IPv6地址，都是很难记忆的。
- 为了使这些地址便于记忆，Internet的设计师们发明了DNS。
- **DNS**将IP地址和域名（一个容易记忆的字符串）联系在一起。
- 当计算机通过域名访问Internet资源时，系统首先通过DNS得到域名对应的IP地址，再通过IP地址访问Internet资源。在这个过程中，IP地址对用户是完全透明的。如果一个域名对应了多个IP地址，DNS从这些IP地址中随机选。



# 域名系统（DNS）

- 域名系统（Domain Name System, DNS）
- DNS将易于记忆的域名与IP地址联系起来

↻ 210.39.3.164 <-> www.szu.edu.cn

- 关系

- ↻ 一台主机可能有多个IP地址
- ↻ 一个IP地址可能被多个域名指向
- ↻ 一个域名可能指向多个IP地址



# InetAddress类

- InetAddress类是Java对IP地址的封装，用于描述IP地址的类。
- 它包括一个主机名和一个IP地址
- 它在java.net包中(rt.jar)，此包中许多类都使用到了InetAddress，包括ServerSocket, Socket, DatagramSocket等。
- 在Java中分别用Inet4Address和Inet6Address类来描述IPv4和IPv6的地址。这两个类都是InetAddress的子类。



# 创建InetAddress对象

- InetAddress没有public的构造方法，要想创建InetAddress对象，必须得依靠它的四个静态方法：
- InetAddress可以通过getLocalHost方法得到本机的InetAddress对象。
- 也可以通过getByName、getAllByName和getByAddress得到远程主机的InetAddress对象。

## ■ 创建方法：

Static InetAddress getByName(String s)

Static InetAddress[] getAllByName(String s)

Static InetAddress getLocalHost()





# getLocalHost方法

- 使用getLocalHost可以得到描述本机IP的InetAddress对象。这个方法的定义：
- `public static InetAddress getLocalHost() throws UnknownHostException`
- 在调用这个方法的程序中捕捉或抛出UnknownHostException这个异常。
- 下面的代码演示了如何使用getLocalHost来得到本机的IP和计算机名。



# getLocalHost方法使用例子

```
import java.net.*;
public class MyInetAddress1 {
    public static void main(String[] args) throws Exception{
        InetAddress localAddress = InetAddress.getLocalHost();
        System.out.println(localAddress);
    }
}
```

## ➤ 运行结果

```
<terminated> MyInetAddress1 [Java Application]
mfq-THINK/172.31.75.47
```

- 当本机绑定了多个IP时，getLocalHost只返回第一个IP。如果想返回本机全部的IP，可以使用getAllByName方法。



# getByName方法

- 可以通过该方法指定域名从DNS中查得与域名相对应的IP地址。
- getByName一个String类型参数，可以通过这个参数指定远程主机的域名，它的定义如下：

```
public static InetAddress getByName(String host) throws UnknownHostException
```

- 如果host所指的域名对应多个IP，getByName返回第一个IP。
- 如果本机名已知，可以使用getByName方法来代替getLocalHost。
- 当host的值是localhost时，返回的IP一般是127.0.0.1。
- 如果host是不存在的域名，getByName将抛出UnknownHostException异常，如果host是IP地址，无论这个IP地址是否存在，getByName方法都会返回这个IP地址（因此getByName并不验证IP地址的正确性）。
- 下面代码演示了如何使用getByName方法：

该方法会与本地DNS服务器建立一个连接,根据提供的名字查找到对应的IP. 首先会在本机缓存中查找,如果能找到,就不会建立网络连接

# getByName方法使用例子

```
import java.net.*;
public class MyInetAddress2 {

    public static void main(String[] args)throws Exception {
        if (args.length == 0)
            return;
        String host = args[0];
        InetAddress address = InetAddress.getByName(host);
        System.out.println(address);
    }
}
```

- 测试1：远程主机www.szu.edu.cn
- 执行如下命令
- java MyInetAddress2 www.szu.edu.cn
- 运行结果：

```
<terminated> MyInetAddress2 [Java Application]
www.szu.edu.cn/210.39.3.164
```



## getByName方法使用例子

- 测试2：本机名mfq-THINK
- 执行如下命令
- java MyInetAddress2 mfq-THINK
- 运行结果：

```
<terminated> MyInetAddress2 [Java Application]
```

```
mfq-THINK/172.31.75.47
```





## getByName方法使用例子

- 测试3：代表本机的localhost
- 执行如下命令
- `java MyInetAddress2 localhost`
- 运行结果：

```
<terminated> MyInetAddress2 [Java Application]  
localhost/127.0.0.1
```



## getByName方法使用例子

- 对于本机来说，除了可以使用本机名或localhost外，还可以在hosts文件中对本机做“IP/域名”映射（在Windows操作系统下）。这个文件在C:\Windows\System32\drivers\etc中。打开hosts，在最后加一行如下所示的字符串：
- 192.168.18.100 www.mysite.com
- 测试4：本机域名www.mysite.com
- 执行如下命令
- java MyInetAddress2 www.mysite.com
- 运行结果：

```
www.mysite.com/192.168.18.100
```



# 创建新的InetAddress对象

- 显示[www.oreilly.com](http://www.oreilly.com)地址：为该域名对应的主机创建一个InetAddress对象

```
OReillyByName.java
1 import java.net.*;
2
3 public class OReillyByName {
4
5     public static void main (String[] args) {
6         try {
7             InetAddress address = InetAddress.getByName("www.oreilly.com");
8             System.out.println(address);
9         } catch (UnknownHostException ex) {
10             System.out.println("Could not find www.oreilly.com");
11         }
12     }
13 }
14
```

```
Console
<terminated> OReillyByName [Java Application] C:\Program Files\Java\jdk1.8.0_
www.oreilly.com/104.86.0.99
```

## getAllByName方法

- 使用getAllByName方法可以从DNS上得到域名对应的所有的IP。这个方法返回一个InetAddress类型的数组。这个方法的定义如下：

```
public static InetAddress[] getAllByName(String host) throws UnknownHostException
```

- 与getByName方法一样，当host不存在时，getAllByName也会抛出UnknownHostException异常，getAllByName也不会验证IP地址是否存在。



# getAllByName方法使用例子

- 下面的代码演示了getAllByName的用法:

```
import java.net.*;
public class MyInetAddress3 {
    public static void main(String[] args) throws Exception{
        if (args.length == 0)
            return;
        String host = args[0];
        InetAddress addresses[] = InetAddress.getAllByName(host);
        for (InetAddress address : addresses)
            System.out.println(address);
    }
}
```

- 测试: 远程主机www.126.com
- 运行结果:

```
<terminated> MyInetAddress3 [Java Application]
www.126.com/125.90.204.122
www.126.com/113.107.57.43
```





## getByAddress方法

- 这个方法必须通过IP地址来创建InetAddress对象，而且IP地址必须是byte数组形式。getByAddress方法有两个重载形式，定义如下：

```
public static InetAddress getByAddress(byte[] addr) throws UnknownHostException
```

```
public static InetAddress getByAddress(String host, byte[] addr) throws  
UnknownHostException
```

- 第一个重载形式只需要传递byte数组形式的IP地址，getByAddress方法并不验证这个IP地址是否存在，只是简单地创建一个InetAddress对象。addr数组的长度必须是4（IPv4）或16（IPv6），如果是其他长度的byte数组，getByAddress将抛出一个UnknownHostException异常。
- 第二个重载形式多了一个host，这个host和getByName、getAllByName方法中的host的意义不同，getByAddress方法并不使用host在DNS上查找IP地址，这个host只是一个用于表示addr的别名。

# getByAddress方法使用例子

- 下面的代码演示了getByAddress的两个重载形式的用法:

```
import java.net.*;
public class MyInetAddress4 {
    public static void main(String[] args) throws Exception{
        byte ip[] = new byte[] { (byte) 141, (byte) 146, 8 , 66};
        InetAddress address1 =    InetAddress.getByAddress(ip);
        InetAddress address2 =
                                InetAddress.getByAddress("Oracle官方网站", ip);
        System.out.println(address1);
        System.out.println(address2);
    }
}
```

- 运行结果:

```
<terminated> MyInetAddress4 [Java Application]
/141.146.8.66
Oracle官方网站/141.146.8.66
```



# 程序安全性问题

- 根据主机名创建一个InetAddress对象不安全。
- InetAddress.getByName()/getAllByName()/getLocalHost()都存在此安全隐患
- 因为这些创建方法需要一个DNS查找,任意的DNS查找会打开一个隐藏的通道,造成信息泄露。
- 在程序中常需先测试一个主机是否支持DNS解析,使用SecurityManager的方法:
- public void checkConnect(String hostname, int port)
- port参数为-1, 该方法能检查能否调用DNS解析指定的hostname



# 为什么不直接通过IP访问网站

➤ 已知: `<terminated> MyInetAddress2 [Java Application]`  
`www.szu.edu.cn/210.39.3.164`

➤ 浏览器中输入: `http://www.szu.edu.cn`



➤ 输入: `http://210.39.3.164`

# 为什么不直接通过IP访问网站

➤ 已知:

<terminated> MyIinetAddress3 [Java Application]

www.126.com/125.90.204.122

www.126.com/113.107.57.43

➤ 浏览器中输入: <http://www.126.com>



➤ 输入: <http://125.90.204.122>



# 为什么不直接通过IP访问网站

- 当在浏览器地址栏中再输入http://www.126.com可以访问，但输入IP http://125.90.204.122却不可以，这并不是客户端的问题，而是服务端对此做了限制。
- 在HTTP协议的请求头有一个Host字段，一般通过http://www.126.com访问服务器时，Host的值就是www.126.com。如果是http://125.90.204.122，那么Host的值就是125.90.204.122。www.126.com的服务器通过检测Host字段防止客户端直接使用IP进行访问。
- 目前有很多网站，如www.sina.com.cn，www.163.com都是这样做的。
- 有一些网站虽然未限制用IP地址来访问，但在使用IP地址访问网站时，却将IP地址又重定位到相应的域名上。如输入http://210.39.3.164会重定位到http://www.szu.edu.cn/szu.asp 上。



# DNS缓存

- 在DNS服务器上查找域名操作非常昂贵：因在通过DNS查找域名的过程中，可能会经过多台中间DNS服务器才能找到指定的域名。
- Java提供DNS缓存解决此问题。
- 当InetAddress类第一次使用某个域名（如www.szu.edu.cn）创建InetAddress对象后，JVM就会将这个域名和它从DNS上获得的信息（如IP地址）都保存在DNS缓存中。
- 当下一次InetAddress类再使用这个域名时，就直接从DNS缓存里获得所需的信息，而无需再访问DNS服务器。
- DNS缓存在默认时将永远保留曾经访问过的域名信息，但我们可以修改这个默认值。



# DNS缓存

- DNS查询的开销较大（可能几秒）
- `InetAddress`会缓存查找的结果
- 默认只缓存10秒
- 本机、本地DNS、其他DNS也会缓存查询，Java无法控制这些缓存
- 修改域名指向可能需要几个小时才能生效
- 按IP地址查找`getByName("x.x.x.x")`，不检查DNS，直接创建对象



# DNS查找结果在java缓存中的保留时间

- 通过java.security.Security.setProperty方法设置系统属性networkaddress.cache.ttl的值（单位：秒）。如下面的代码将指定成功的DNS查找结果在Java缓存中保留的时间为10秒：

```
java.security.Security.setProperty("networkaddress.cache.ttl", 10);
```

- 设置系统属性networkaddress.cache.negative.ttl的值，指定不成功的DNS查找结果在Java缓存中保留的时间（秒数）。
- 如果将networkaddress.cache.ttl属性值设为-1，那么DNS在InetAddress类中的缓存数据将永不过期。



```

import java.net.*;
public class MyDNS {
    public static void main(String[] args) throws Exception{
        // args[0]: 本机名 args[1]: 缓冲时间
        if (args.length < 2)
            return;
        java.security.Security.setProperty("networkaddress.cache.ttl", args[1]);
        long time = System.currentTimeMillis();
        InetAddress addresses1[] = InetAddress.getAllByName(args[0]);
        System.out.println("addresses1:  "
            + String.valueOf(System.currentTimeMillis() - time)
            + "毫秒");
        for (InetAddress address : addresses1)
            System.out.println(address);
        System.out.print("按任意键继续 ...");
        System.in.read();
        time = System.currentTimeMillis();
        InetAddress addresses2[] = InetAddress.getAllByName(args[0]);
        System.out.println("addresses2:  "
            + String.valueOf(System.currentTimeMillis() - time));
        for (InetAddress address : addresses2)
            System.out.println(address);
    }
}

```

缓冲超时参数

暂停程序

使用同一个域名(args[0])再建立一个InetAddress数组

当用户等待一段时间后，可以按任意键继续，并使用同一个域名(args[0])再建立一个InetAddress数组。如果用户等待的这段时间比DNS缓存超时小，那么无论情况如何变化，addresses2和addresses1数组中的元素是一样的，并且创建addresses2数组所花费的时间一般为0毫秒（小于1毫秒后，Java无法获得更精确的时间）。

# 修改DNS缓存编程例子

- 测试1：执行如下命令（将DNS缓存超时设为5秒）：
- java MyDNS www.126.com 5
- 运行结果1（在5秒之内按任意键）：

```
<terminated> MyDNS [Java Application]  
addresses1:    18毫秒  
www.126.com/125.90.204.122  
www.126.com/113.107.57.43  
按任意键继续...  
addresses2:    0毫秒  
www.126.com/125.90.204.122  
www.126.com/113.107.57.43
```

- 运行结果2（在5秒后按任意键）：

```
<terminated> MyDNS [Java Application]  
addresses1:    20毫秒  
www.126.com/125.90.204.122  
www.126.com/113.107.57.43  
按任意键继续...  
addresses2:    1毫秒  
www.126.com/125.90.204.122  
www.126.com/113.107.57.43
```

测试1可能出现两个运行结果。如果在出现“按任意键继续...”后，在5秒之内按任意键继续后，就会得到运行结果1，从这个结果可以看出，addresses2所用的时间为0毫秒，也就是说，addresses2并未真正访问DNS服务器，而是直接从内存中的DNS缓存得到的数据。当在5秒后按任意键继续后，就会得到运行结果2，这时，内存中的DNS缓存中的数据已经释放，所以addresses2还得再访问DNS服务器，因此，addresses2的时间是1毫秒

# 修改DNS缓存编程例子

- 测试2：执行如下命令（mfq-THINK为本机的计算机名，DNS缓存超时设为永不过期[-1]）：
- java MyDNS mfq-THINK -1
- 运行结果1：

```
<terminated> MyDNS [Java Application] C:\Program Files\Java
addresses1:    13毫秒
mfq-THINK/172.31.75.47
mfq-THINK/192.168.18.1
mfq-THINK/fe80:0:0:0:5c31:34b3:2f97:fa79%20
按任意键继续...
addresses2:    0毫秒
mfq-THINK/172.31.75.47
mfq-THINK/192.168.18.1
mfq-THINK/fe80:0:0:0:5c31:34b3:2f97:fa79%20
```

将DNS缓存设为永不过期后，addresses2是从DNS缓存中得到数据。无论过多少时间，按任意键后，addresses2仍然得到了与address1一样的IP地址，而且addresses2的时间是0毫秒。

如果域名在DNS服务器上不存在，那么客户端在进行一段时间的尝试后（平均为5秒），就会抛出一个UnknownHostException异常。为了让下一次访问这个域名时不再等待，DNS缓存将这个错误信息也保存了起来。

只有第一次访问错误域名时才进行5秒左右的尝试，以后再访问这个域名时将直接抛出UnknownHostException异常，而无需再等待5秒钟。



## 域名错误信息的保留时间设置

- 访问域名失败的原因可能是这个域名真的不存在，也可能是因为DNS服务器或是其他的硬件或软件的临时故障，因此，一般不能将这个域名错误信息一直保留。
- 可以通过`networkaddress.cache.negative.ttl`属性设置保留这些信息的时间，这个属性的默认值是10秒。
- 也可以通过`java.security.Security.setProperty`方法或`java.security`文件来设置。
- 下面的代码演示了`networkaddress.cache.negative.ttl`属性的用法。
- 程序分别测试了`address1`和`address2`访问 **www.123456.com**（这是个不存在的域名）后，用了多长时间抛出`UnknownHostException`异常



```
import java.net.*;
public class MyDNS1 {
    public static void main(String[] args) throws Exception {
        java.security.Security.setProperty("networkaddress.cache.negative.ttl", "5");
        long time = 0;
        try
        {
            time = System.currentTimeMillis();
            InetAddress.getByName("www.123456.com");
        }
        catch (Exception e)
        {
            System.out.println("www.123456.com不存在! address1: "
                + String.valueOf(System.currentTimeMillis() - time) + "毫秒");
        }
        //Thread.sleep(6000); // 延迟6秒
        try
        {
            time = System.currentTimeMillis();
            InetAddress.getByName("www.123456.com");
        }
        catch (Exception e)
        {
            System.out.println("www.123456.com不存在! address2: "
                + String.valueOf(System.currentTimeMillis() - time) + "毫秒");
        }
    }
}
```

将属性值设计为5秒

# 域名错误信息的保留时间设置例子

## ➤ 测试1：运行结果

```
<terminated> MyDNS1 [Java Application] C:\Program I  
www.123456.com不存在! address1: 16毫秒  
www.123456.com不存在! address2: 0毫秒  
|
```

- 分析： address1用了16毫秒抛出了异常， address2用了0毫秒就抛出了异常。可以断定address2是从DNS缓存里获得了域名www123456.com不可访问的信息，所以就直接抛出了UnknownHostException异常。
- 测试2： 如果将上面代码中的延迟代码的注释去掉，那么可能得到如下的运行结果：

```
<terminated> MyDNS1 [Java Application] C:\Program  
www.123456.com不存在! address1: 92毫秒  
www.123456.com不存在! address2: 22毫秒
```

- 分析： 在第6秒时，DNS缓存中的数据已经被释放，因此，address2仍需要访问DNS服务器才能知道www.123456com是不可访问的域名。

## 域名错误信息的保留时间设置原则

- 总结注意事项:
- 可以根据实际情况设置`networkaddress.cache.ttl`属性的值。一般将这个属性的值设为-1。但如果访问的是动态映射的域名（如使用动态域名服务将域名映射成ADSL的动态IP），就可能产生IP地址变化后，客户端得到的还是原来的IP地址的情况。
- 在设置`networkaddress.cache.negative.ttl`属性值时最好不要将它设为-1，否则如果一个域名因为暂时的故障而无法访问，那么程序再次访问这个域名时，即使这个域名恢复正常，程序也无法再访问这个域名了。除非重新运行程序。



## InetAddress的获取方法

- 四个获取方法可以将主机名作为字符串返回，将IP地址返回为字符串和字节数组。
- **String getHostName()**
  - ∞ 返回主机名和IP
- **String getCanonicalHostName()**
  - ∞ 查询DNS，再返回主机名和IP
- **Byte[] getAddress()**
  - ∞ 返回IP，字节类型，IPv4或IPv6
- **String.getHostAddress()**
  - ∞ 返回IP，字符串类型，IPv4或IPv6



# Inet4Address/Inet6Address

- public final Inet4Address extends InetAddress
- public final Inet6Address extends InetAddress
- 一般用不到，可用`getAddress()`返回的数组长度判断IP地址类型

```
AddressTests.java ✕  
1 import java.net.*;  
2  
3 public class AddressTests {  
4  
5     public static int getVersion(InetAddress ia) {  
6         byte[] address = ia.getAddress();  
7         if (address.length == 4) return 4;  
8         else if (address.length == 16) return 6;  
9         else return -1;  
10    }  
11 }
```





# 使用getHostName方法获得域名

- getHostName方法可以得到远程主机的域名，也可以得到本机名，定义如下：

```
public String getHostName()
```

- 三种创建InetAddress对象的方式，getHostName返回的值是不同的。
- 1.使用getLocalHost方法创建InetAddress对象：  
getHostName返回的是本机名。

```
InetAddress address = InetAddress.getLocalHost();  
System.out.println(address.getHostName()); // 输出本机名
```

- 2.使用域名创建InetAddress对象：用域名作为getByName和getAllByName方法的参数调用这两个方法后，系统会自动记住这个域名。当调用getHostName方法时，就无需再访问DNS服务器，而是直接将这个域名返回。

```
InetAddress address = InetAddress.getByName("www.szu.edu.cn");  
System.out.println(address.getHostName()); // 无需访问DNS服务器，直接返回域名
```

# 使用getHostName方法获得域名

- 3.使用IP地址创建InetAddress对象：使用IP地址创建InetAddress对象时（getByName、getAllByName和getByAddress方法都可以通过IP地址创建InetAddress对象），并不需要访问DNS服务器。通过DNS服务器查找域名的工作就由getHostName方法来完成。如果这个IP地址不存在或DNS服务器不允许进行IP地址和域名的映射，getHostName方法就直接返回这个IP地址。

```
InetAddress address = InetAddress.getByName("141.146.8.66");  
System.out.println(address.getHostName()); // 需要访问DNS服务器才能得到域名  
InetAddress address = InetAddress.getByName("1.2.3.4"); // IP地址不存在  
System.out.println(address.getHostName()); // 直接返回IP地址
```

- 总结：三种方式，只有通过使用IP地址创建的InetAddress对象调用getHostName方法时才访问DNS服务器。在其他情况，getHostName方法并不会访问DNS服务器，而是直接将域名或本机名返回。
- 下面的代码演示了在不同情况下如何使用getHostName方法，并计算了各种情况所需的毫秒数。

```
import java.net.*;
public class DomainName {
    public static void main(String[] args) throws Exception{
        long time = 0;
        // 得到本机名
        InetAddress address1 = InetAddress.getLocalHost();
        System.out.println("本机名:  " + address1.getHostName());
        // 直接返回域名
        InetAddress address2 =InetAddress.getByName("www.szu.edu.cn");
        time = System.currentTimeMillis();
        System.out.print("直接得到域名:  " + address2.getHostName());
        System.out.println("  所用时间: "
+ String.valueOf(System.currentTimeMillis() - time) + " 毫秒");
        // 通过DNS查找域名
        InetAddress address3 = InetAddress.getByName("210.39.3.164");
        System.out.println("address3:  " + address3);  // 域名为空
        time = System.currentTimeMillis();
        System.out.print("通过DNS查找域名:  "+address3.getHostName());
        System.out.println("  所用时间: " +
String.valueOf(System.currentTimeMillis() - time)+ " 毫秒");
        System.out.println("address3:  " + address3);  // 同时输出域名
和IP地址
    }
}
```

# 使用getHostName方法获得域名例子

## ➤ 运行结果:

```
<terminated> DomainName (1) [Java Application] C:\Program Files\Java  
本机名: mfg-THINK  
直接得到域名: www.szu.edu.cn 所用时间: 0 毫秒  
address3: /210.39.3.164  
通过DNS查找域名: www.szu.edu.cn 所用时间: 9119 毫秒  
address3: www.szu.edu.cn/210.39.3.164
```

- 分析: 第一个毫秒数是0, 而第二个毫秒数是92。
- 说明使用域名创建的InetAddress对象在使用getHostName方法时并不访问DNS服务器。
- 而使用IP地址创建的InetAddress对象在使用getHostName方法时需要访问DNS服务器。



# 使用getCanonicalHostName方法获得主机名

- getCanonicalHostName方法和getHostName方法功能一样，也是得到远程主机的域名。
- 但getCanonicalHostName得到的是主机名，而getHostName得到的主机别名。方法定义：

```
public String getCanonicalHostName()
```

- 该方法的返回值与InetAddress对象创建方式有关，分三种情况：
- 一是，若InetAddress对象是使用getLocalHost创建的，则该方法的返回值与getHostName方法得到的一样，都是本机名。
- 二是，若InetAddress对象是使用IP地址创建的，则该方法的返回值与getHostName方法得到的一样，它们的值可能是主机名，也可能是IP地址。
- 三是，若InetAddress对象是使用域名创建的，在创建InetAddress对象时，主机名和主机别名若已确定，则该方法不会访问DNS服务器，直接返回主机别名。否则，则会访问DNS服务器，则该方法的返回值取决于DNS服务器设置。

# 使用getCanonicalHostName方法例子

```
import java.net.*;
public class DomainName2 {
    public static void outHostName(InetAddress address, String s)
    {
        System.out.println("通过" + s + "创建InetAddress对象");
        System.out.println("主 机 名:" + address.getCanonicalHostName());
        System.out.println("主机别名:" + address.getHostName());
        System.out.println("");
    }

    public static void main(String[] args) throws Exception{
        outHostName(InetAddress.getLocalHost(), "getLocalHost方法");
        outHostName(InetAddress.getByName("www.ibm.com"), "www.ibm.com");
        outHostName(InetAddress.getByName("www.126.com"), "www.126.com");
        outHostName(InetAddress.getByName("202.108.9.77"), "202.108.9.77");
        outHostName(InetAddress.getByName("211.100.26.121"),
"211.100.26.121");
    }
}
```



# 使用getCanonicalHostName方法例子

## ➤ 运行结果:

```
<terminated> DomainName2 [Java Application] C:\Progra
```

```
通过getLocalHost方法创建InetAddress对象
```

```
主机名:mfq-THINK
```

```
主机别名:mfq-THINK
```

```
通过www.ibm.com创建InetAddress对象
```

```
主机名:121.205.160.215
```

```
主机别名:www.ibm.com
```

```
通过www.126.com创建InetAddress对象
```

```
主机名:125.90.204.122
```

```
主机别名:www.126.com
```

```
|
```

```
通过202.108.9.77创建InetAddress对象
```

```
主机名:zz-9-77-a8.bta.net.cn
```

```
主机别名:zz-9-77-a8.bta.net.cn
```

```
通过211.100.26.121创建InetAddress对象
```

```
主机名:211.100.26.121
```

```
主机别名:211.100.26.121
```

# 使用getHostAddress方法获得IP地址

```
import java.net.*;
public class GetIP{
    public static void main(String[] args) throws Exception{
        // 输出IPv4地址
        InetAddress ipv4Address1 = InetAddress.getByName("1.2.3.4");
        System.out.println("ipv4Address1: " + ipv4Address1.getHostAddress());
        InetAddress ipv4Address2 = InetAddress.getByName("www.ibm.com");
        System.out.println("ipv4Address2: " + ipv4Address2.getHostAddress());
        InetAddress ipv4Address3 = InetAddress.getByName("www.szu.edu.cn");
        System.out.println("ipv4Address3: " + ipv4Address3.getHostAddress());
        // 输出IPv6地址
        InetAddress ipv6Address1 = InetAddress.getByName("abcd:123::22ff");
        System.out.println("ipv6Address1: " + ipv6Address1.getHostAddress());
        InetAddress ipv6Address2 = InetAddress.getByName("www.neu6.edu.cn");
        System.out.println("ipv6Address2: " + ipv6Address2.getHostAddress());
        // 输出本机全部的IP地址
        InetAddress Addresses[] = InetAddress.getAllByName("www.szu.edu.cn");
        for (InetAddress address : Addresses)
            System.out.println("本机地址: " + address.getHostAddress());
    }
}
```

# 使用getHostAddress方法获得IP地址

➤ 运行结果:

```
<terminated> GetIP [Java Application] C:\Program Files\J  
ipv4Address1: 1.2.3.4  
ipv4Address2: 183.61.92.117  
ipv4Address3: 210.39.3.164  
ipv6Address1: abcd:123:0:0:0:0:0:22ff  
ipv6Address2: 2001:da8:9000:0:0:0:0:7  
本机地址: 210.39.3.164
```



# 使用getAddress方法获得IP地址

- getAddress方法和getHostAddress类似，它们区别是getHostAddress方法返回的是字符串形式的IP地址，而getAddress方法返回的是byte数组形式的IP地址。方法定义：

```
public byte[] getAddress()
```

- 这个方法返回的byte数组是有符号的。
- 在Java中byte类型的取值范围是-128~127。
- 如果返回的IP地址的某个字节是大于127的整数，在byte数组中就是负数。
- 由于Java中没有无符号byte类型，因此，要想显示正常的IP地址，必须使用int或long类型。
- 下面代码演示了如何利用getAddress返回IP地址，以及如何将IP地址转换成正整数形式。

# 使用getAddress方法获得IP地址

```
import java.net.*;
public class GetIP2{
    public static void main(String[] args) throws Exception{
        InetAddress address = InetAddress.getByName("www.csdn.net");
        byte ip[] = address.getAddress();
        for (byte ipSegment : ip)
            System.out.print(ipSegment + " ");
        System.out.println("");
        for (byte ipSegment : ip)
        {
            int newIPSegment = (ipSegment < 0) ? 256 + ipSegment :
ipSegment;
            System.out.print(newIPSegment + " ");
        }
    }
}
```

<terminated> GetIP2 [Java Application]

101 -55 -84 -27

101 201 172 229 |

## ➤ 运行结果

- 分析：第一行输出了未转换的IP地址，由于www.csdn.net的IP地址的第一个字节大于127，因此，输出了一个负数。而第二行由于将IP地址的每一个字节转换成了int类型，因此，输出了正常的IP地址。

# 获取方法

- 给定地址，找出主机别名:先得到InternetAddress对象,再转换得到主机别名

```
ReverseTest.java
1 import java.net.*;
2
3 public class ReverseTest {
4
5     public static void main (String[] args) throws UnknownHostException {
6         InetAddress ia = InetAddress.getByName("104.86.0.99");
7         System.out.println(ia.getCanonicalHostName());
8     }
9 }
```

```
Console
<terminated> ReverseTest [Java Application] C:\Program Files\Java\jdk1.8.0_1
a104-86-0-99.deploy.static.akamaitechnologies.com
```





# 测试IP地址的性质

- 有时程序需要测试IP地址的性质:使用10个方法测试从命令行输入的一个地址的性质
- 演示示例代码ch4.IPCharacteristics.java ,ex4-6

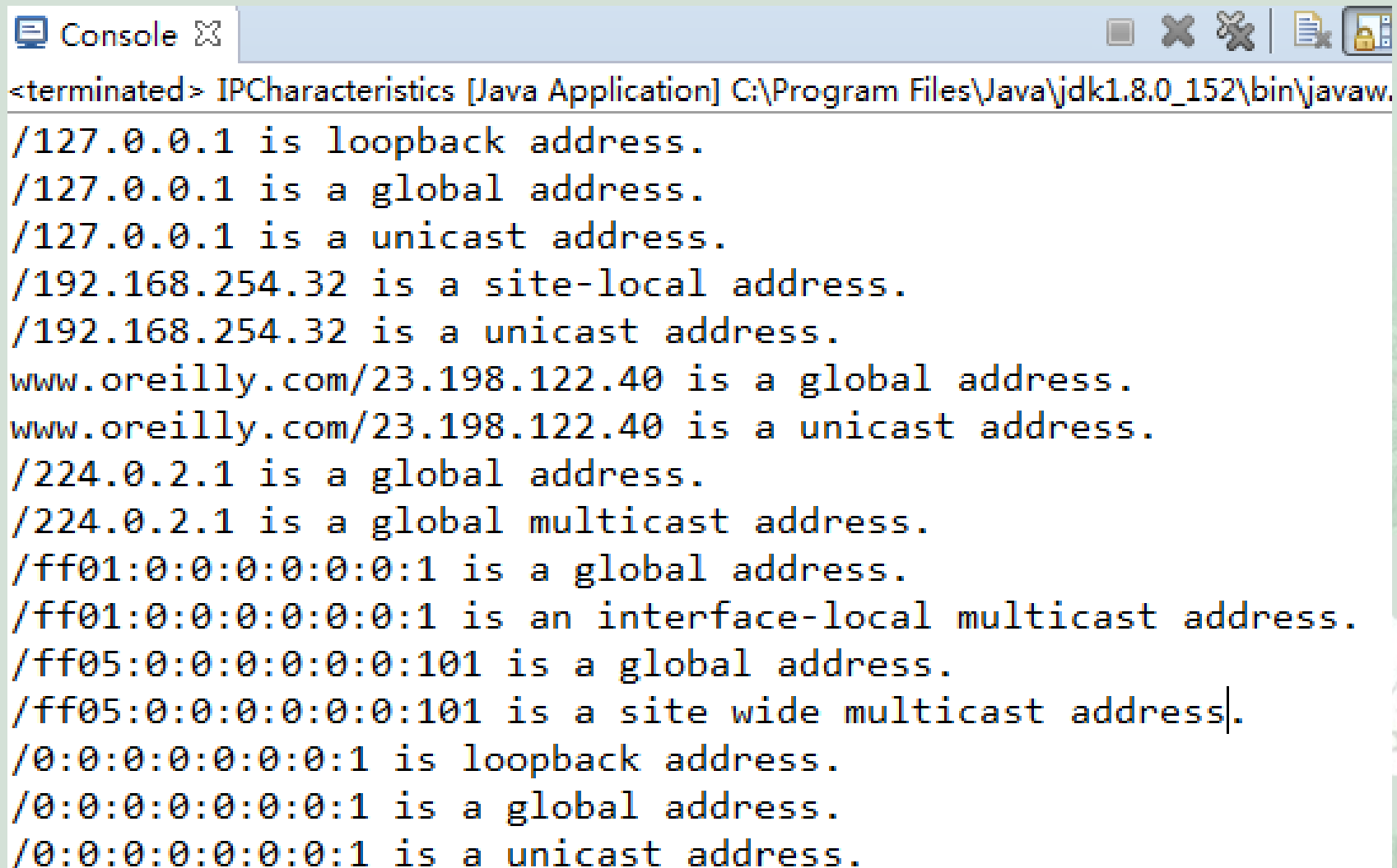
IPCharacteristics.java

```
2 import java.net.*;
3 public class IPCharacteristics {
4     public static void main(String[] args) {
5         for( String str: args) {
6             try {
7                 //InetAddress address = InetAddress.getBy_name(args[0]);
8                 InetAddress address = InetAddress.getBy_name(str);
9                 //如果是通配地址,则返回true
10                if (address.isAnyLocalAddress()) {
11                    System.out.println(address + " is a wildcard address.");
12                }
13                //如果是回送地址,则返回true
14                if (address.isLoopbackAddress()) {
15                    System.out.println(address + " is loopback address.");
16                }
17                //如果是一个IPV6本地链接地址,则返回true
18                if (address.isLinkLocalAddress()) {
19                    System.out.println(address + " is a link-local address.");
20                } else if (address.isSiteLocalAddress()) { //如果是一个IPV6本地网站地址,则返回true
21                    System.out.println(address + " is a site-local address.");
22                } else {
23                    System.out.println(address + " is a global address.");
24                }
25            } catch (Exception e) {
26                System.out.println("Error: " + e.getMessage());
27            }
28        }
29    }
30 }
```

```
25     if (address.isMulticastAddress()) { // 如果是一个组播地址，则返回true
26         if (address.isMCGlobal()) { // 如果是全球组播地址，则返回true
27             System.out.println(address + " is a global multicast address.");
28         } else if (address.isMCOrgLocal()) { // 如果是一个组织范围组播地址，则返回true
29             System.out.println(address
30                 + " is an organization wide multicast address.");
31         } else if (address.isMCSiteLocal()) { // 如果是一个网站范围组播地址，则返回true
32             System.out.println(address + " is a site wide multicast address.");
33         } else if (address.isMCLinkLocal()) { // 如果是一个子网范围组播地址，则返回true
34             System.out.println(address + " is a subnet wide multicast address.");
35         } else if (address.isMCNodeLocal()) { // 如果是一个本地接口组播地址，则返回true
36             System.out.println(address
37                 + " is an interface-local multicast address.");
38         } else {
39             System.out.println(address + " is an unknown multicast address type.");
40         }
41     } else {
42         System.out.println(address + " is a unicast address.");
43     }
44 } catch (UnknownHostException ex) {
45     System.err.println("Could not resolve " + args[0]);
46 }
47 }
48 }
49 }
```

# 测试IP地址的性质

直接运行IPCharacteristics类，不带参数，控制台输出：



The screenshot shows a Java console window titled "Console" with a standard Windows-style title bar. The text inside the console is the output of a Java application named "IPCharacteristics". The output lists various IP addresses and their properties, such as whether they are loopback, global, unicast, or multicast addresses. The text is displayed in a monospaced font, typical of a terminal or console window.

```
<terminated> IPCharacteristics [Java Application] C:\Program Files\Java\jdk1.8.0_152\bin\javaw.  
/127.0.0.1 is loopback address.  
/127.0.0.1 is a global address.  
/127.0.0.1 is a unicast address.  
/192.168.254.32 is a site-local address.  
/192.168.254.32 is a unicast address.  
www.oreilly.com/23.198.122.40 is a global address.  
www.oreilly.com/23.198.122.40 is a unicast address.  
/224.0.2.1 is a global address.  
/224.0.2.1 is a global multicast address.  
/ff01:0:0:0:0:0:0:1 is a global address.  
/ff01:0:0:0:0:0:0:1 is an interface-local multicast address.  
/ff05:0:0:0:0:0:0:101 is a global address.  
/ff05:0:0:0:0:0:0:101 is a site wide multicast address|  
/0:0:0:0:0:0:0:1 is loopback address.  
/0:0:0:0:0:0:0:1 is a global address.  
/0:0:0:0:0:0:0:1 is a unicast address.
```

# NetworkInterface

- 表示物理网卡或虚拟网卡

- ⌘ static NetworkInterface getByName(String name)

- 列出所有网卡

- ⌘ static Enumeration getInetAddresses()



```
import java.net.*;
import java.util.*;
public class InterfaceList {
    public static void main(String[] args) throws SocketException{
        try{
            Enumeration<NetworkInterface> interfaces =
NetworkInterface.getNetworkInterfaces();
            while(interfaces.hasMoreElements()){
                NetworkInterface ni = interfaces.nextElement();
                //System.out.println("-----");
                System.out.println(ni);
                //System.out.println("UP: " + ni.isUp());
            }
        }
        catch(Exception e){
            System.out.println(e);
        }
    }
}
```

<terminated> InterfaceList [Java Application] C:\Program Files\Java\jdk1.7.0\_07\bin

```
name:lo (Software Loopback Interface 1)
name:eth0 (WAN Miniport (Network Monitor))
name:net0 (Bluetooth Device (RFCOMM Protocol TDI))
name:ppp0 (WAN Miniport (PPPOE))
name:net1 (Teredo Tunneling Pseudo-Interface)
name:net2 (Microsoft ISATAP Adapter #2)
name:net3 (Microsoft ISATAP Adapter)
name:net4 (WAN Miniport (IKEv2))
name:net5 (WAN Miniport (L2TP))
name:net6 (WAN Miniport (SSTP))
name:eth1 (Intel(R) 82577LM Gigabit Network Connection)
name:eth2 (VMware Virtual Ethernet Adapter for VMnet8)
name:net7 (WAN Miniport (PPTP))
name:net8 (Intel(R) WiFi Link 1000 BGN)
name:eth3 (Bluetooth ???? (????????ò??))
name:eth4 (Microsoft Kernel Debug Network Adapter)
name:ppp1 (RAS Async Adapter)
name:eth5 (WAN Miniport (IP))
name:net9 (Microsoft ISATAP Adapter #4)
name:eth6 (VMware Virtual Ethernet Adapter for VMnet1)
name:eth7 (WAN Miniport (IPv6))
name:eth8 (Intel(R) 82577LM Gigabit Network Connection-WFP Native MAC Layer LightWeight Filter-0000)
name:eth9 (Intel(R) 82577LM Gigabit Network Connection-QoS Packet Scheduler-0000)
name:eth10 (Intel(R) 82577LM Gigabit Network Connection-WFP 802.3 MAC Layer LightWeight Filter-0000)
name:net10 (Intel(R) WiFi Link 1000 BGN-WFP Native MAC Layer LightWeight Filter-0000)
name:net11 (Intel(R) WiFi Link 1000 BGN-Virtual WiFi Filter Driver-0000)
name:net12 (Intel(R) WiFi Link 1000 BGN-Native WiFi Filter Driver-0000)
name:net13 (Intel(R) WiFi Link 1000 BGN-QoS Packet Scheduler-0000)
name:net14 (Intel(R) WiFi Link 1000 BGN-WFP 802.3 MAC Layer LightWeight Filter-0000)
name:eth11 (WAN Miniport (IP)-WFP Native MAC Layer LightWeight Filter-0000)
name:eth12 (WAN Miniport (IP)-QoS Packet Scheduler-0000)
name:eth13 (WAN Miniport (IPv6)-WFP Native MAC Layer LightWeight Filter-0000)
name:eth14 (WAN Miniport (IPv6)-QoS Packet Scheduler-0000)
name:eth15 (WAN Miniport (Network Monitor)-WFP Native MAC Layer LightWeight Filter-0000)
name:eth16 (WAN Miniport (Network Monitor)-QoS Packet Scheduler-0000)
```

的的有网络接口

结果:



## 应用编程思考？

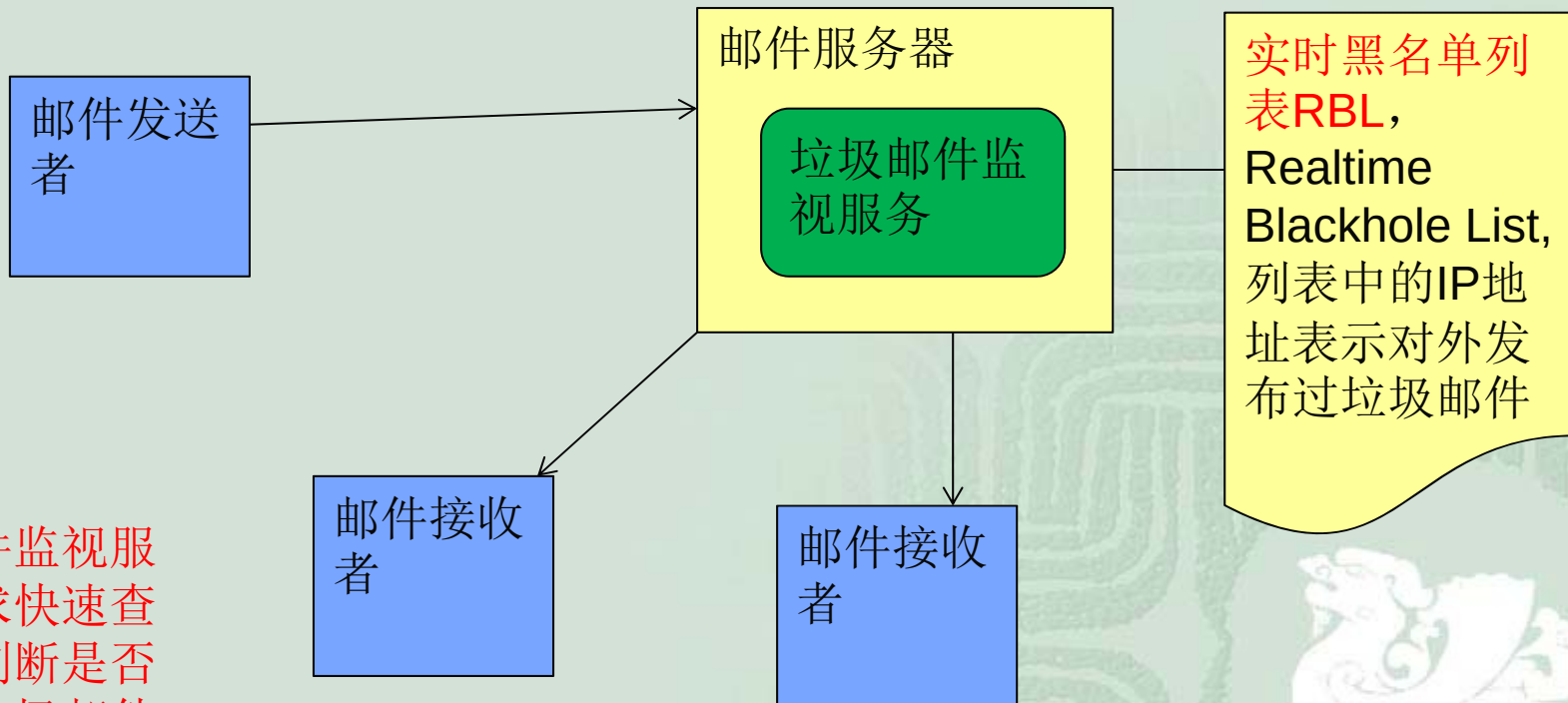
编程发现你的主机在局域网（深大校园网）内的IP。对比分析程序执行的结果与使用ipconfig命令查看本机ip的结果。尝试在有线连接和无线连接网络情况下，分别执行你的“发现本机IP程序”，观察分析结果有无何不同？

思考：当本机环境复杂时，比如有多张网卡、一个网络接口配置了多个IP等，如何获得想要的IP？



# InetAddress的应用: SpamCheck

- 应用问题: 服务器编程时, 如何快速检查判断地址是否是一个已知的垃圾邮件发送者?



垃圾邮件监视服务: 需求快速查询响应判断是否是一个垃圾邮件发送请求。



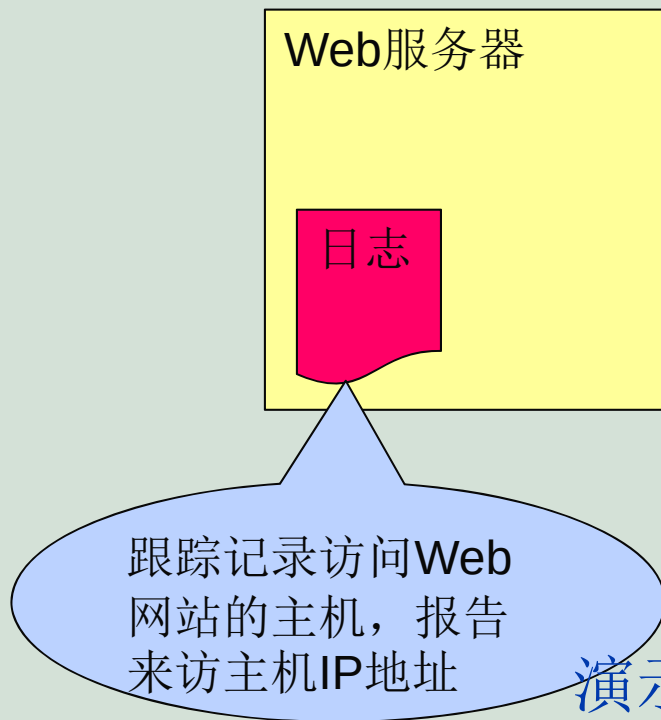
# InetAddress的应用: SpamCheck

- **快速**检查判断地址是否是一个已知的垃圾邮件发送者的流程:
- 查看一个IP地址是否是一个已知的垃圾邮件发送者:
  - 1. 逆置此地址的字节 (例如要查看IP207.34.56.23)
  - 2. 增加黑洞服务的域 ( 23.56.34.207.sbl.spamhaus.org )
  - 3. 查找此地址 ( 23.56.34.207.sbl.spamhaus.org )
  - 4. 判断: 若找到此地址, 说明是一个垃圾邮件发送者, 否则, 说明它不是。
- 演示运行示例程序ch4.SpamCheck.java 向 sbl.spamhaus.org 询问某些IP地址 (例如 207.34.56.23 等) 是否是一个垃圾邮件发送者。

```
2 import java.net.*;
3 public class SpamCheck {
4     public static final String BLACKHOLE = "sbl.spamhaus.org";
5     public static void main(String[] args) throws UnknownHostException {
6         for (String arg: args) {
7             if (isSpammer(arg)) {
8                 System.out.println(arg + " is a known spammer.");
9             } else {
10                 System.out.println(arg + " appears legitimate.");
11             }
12         }
13     }
14     private static boolean isSpammer(String arg) {
15         try {
16             InetAddress address = InetAddress.getByName(arg);
17             byte[] quad = address.getAddress();
18             String query = BLACKHOLE;
19             for (byte octet : quad) {
20                 int unsignedByte = octet < 0 ? octet + 256 : octet;
21                 query = unsignedByte + "." + query;
22             }
23             InetAddress.getByAddress(query);
24             return true;
25         } catch (UnknownHostException e) {
26             return false;
27         }
28     }
}
```

# InetAddress的应用：离线处理日志文件

- 应用问题：如何通过快速离线处理日志文件来提升Web服务器的性能？



在分析日志时，需要将来访主机IP转换为主机名，例如，进行简单的日志分析处理：读取日志文件，显示各行时，将IP地址转换为主机名。

如何编程处理？

演示运行示例ch4.Weblog.java, ex4-10  
分析程序的类结构

日志记录项非常多，数百万行。程序的弊端？如何改进程序设计，提升性能。



```
2 import java.io.*;
3 import java.net.*;
4 public class Weblog {
5     public static void main(String[] args) {
6         try (FileInputStream fin = new FileInputStream(args[0]);
7             Reader in = new InputStreamReader(fin);
8             BufferedReader bin = new BufferedReader(in);) {
9             for (String entry = bin.readLine();
10                 entry != null;
11                 entry = bin.readLine()) {
12                 // separate out the IP address
13                 int index = entry.indexOf(' ');
14                 String ip = entry.substring(0, index);
15                 String theRest = entry.substring(index);
16                 // Ask DNS for the hostname and print it out
17                 try {
18                     InetAddress address = InetAddress.getByName(ip);
19                     System.out.println(address.getHostName() + theRest);
20                 } catch (UnknownHostException ex) {
21                     System.err.println(entry);
22                 }
23             }
24         } catch (IOException ex) {
25             System.out.println("Exception: " + ex);
26         }
27     }
28 }
```



# Web服务器日志文件的快速处理

- 使用多线程？为每一条日志记录项生成一个处理线程？拖垮服务器！
- 使用线程池

示例ch4.LookupTask.java, PooledWeblog.java, ex4-11,ex4-12  
分析程序的类结构



LookupTask.java

```
3 import java.net.*;
4 import java.util.concurrent.Callable;
5
6 public class LookupTask implements Callable<String> {
7
8     private String line;
9
10    public LookupTask(String line) {
11        this.line = line;
12    }
13
14    @Override
15    public String call() {
16        try {
17            // separate out the IP address
18            int index = line.indexOf(' ');
19            String address = line.substring(0, index);
20            String theRest = line.substring(index);
21            String hostname = InetAddress.getByName(address).getHostName();
22            return hostname + " " + theRest;
23        } catch (Exception ex) {
24            return line;
25        }
26    }
27 }
```

```
2 import java.io.*;
3 import java.util.*;
4 import java.util.concurrent.*;
5 public class PooledWeblog {
6     private final static int NUM_THREADS = 4;
7     public static void main(String[] args) throws IOException {
8         ExecutorService executor = Executors.newFixedThreadPool(NUM_THREADS);
9         Queue<LogEntry> results = new LinkedList<LogEntry>();
10
11         try (BufferedReader in = new BufferedReader(
12             new InputStreamReader(new FileInputStream(args[0]), "UTF-8")) {
13             for (String entry = in.readLine(); entry != null; entry = in.readLine()) {
14                 LookupTask task = new LookupTask(entry);
15                 Future<String> future = executor.submit(task);
16                 LogEntry result = new LogEntry(entry, future);
17                 results.add(result);
18             }
19         }
```



```
20 // Start printing the results. This blocks each time a result isn't ready.
21 for (LogEntry result : results) {
22     try {
23         System.out.println(result.future.get());
24     } catch (InterruptedException | ExecutionException ex) {
25         System.out.println(result.original);
26     }
27 }
28
29 executor.shutdown();
30 }
31
32 private static class LogEntry {
33     String original;
34     Future<String> future;
35
36     LogEntry(String original, Future<String> future) {
37         this.original = original;
38         this.future = future;
39     }
40 }
41 }
```

# 编程思考1

- 使用线程池处理服务器日志程序有何设计缺点？
- 若让你编程处理服务器上的日志文件，该如何设计程序？需考虑日志记录项的快速写入，快速读出分析处理，成千上万、甚至百万条日志记录文件的打开占用大量内存问题。试写一个日志处理服务程序。



## 编程思考2

- 试尝试使用log4j开源插件，编程实现Web服务器日志功能，主要实现日志信息的格式化定向输出存储（日志文件中）。





谢谢

